

Chapter 2

Database System Concepts and Architecture



5th Edition

Elmasri / Navathe

Data Models

■ Data Model:

- is a set of rules and ideas used to:
- describe how the database is structured?
- explain how data can be used or changed?
- define rules (constraints) that the data must follow

■ Data Model Structure and Constraints:

- Constructs are used to define the database structure
- Constructs typically include ***elements*** (and their ***data types***) as well as groups of elements (e.g. ***record, table***), and ***relationships*** among such groups
- Constraints specify some restrictions on valid data; these constraints must be enforced at all times

Data Models (continued)

■ Data Model Operations:

- Operations on the data model may include ***basic model operations*** (e.g. generic insert, delete, update) and ***user-defined operations*** (e.g. compute_student_gpa, update_inventory)

Schemas versus Instances

- Database Schema:
 - Full description of a database — structure, data types, and constraints.
- Schema Diagram:
 - A visual illustration of the database schema.
- Schema Construct:
 - A ***component*** of the schema or an object within the schema, e.g., STUDENT, COURSE.

Schemas versus Instances

- Database State:

- The actual data stored in the database at a specific moment.
- Other names: Database **instance**, **snapshot**, or **occurrence**.
- Also called database instance (or occurrence or snapshot).
 - The term *instance* is also applied to individual database components, e.g. *record instance*, *table instance*, *entity instance*

Database Schema vs. Database State

- Database State:
 - Refers to the ***content*** of a database at a moment in time.
- Initial Database State:
 - Refers to the database state when it is initially loaded into the system.
- Valid State:
 - A state that satisfies the structure and constraints of the database.

Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

Example of a database state

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

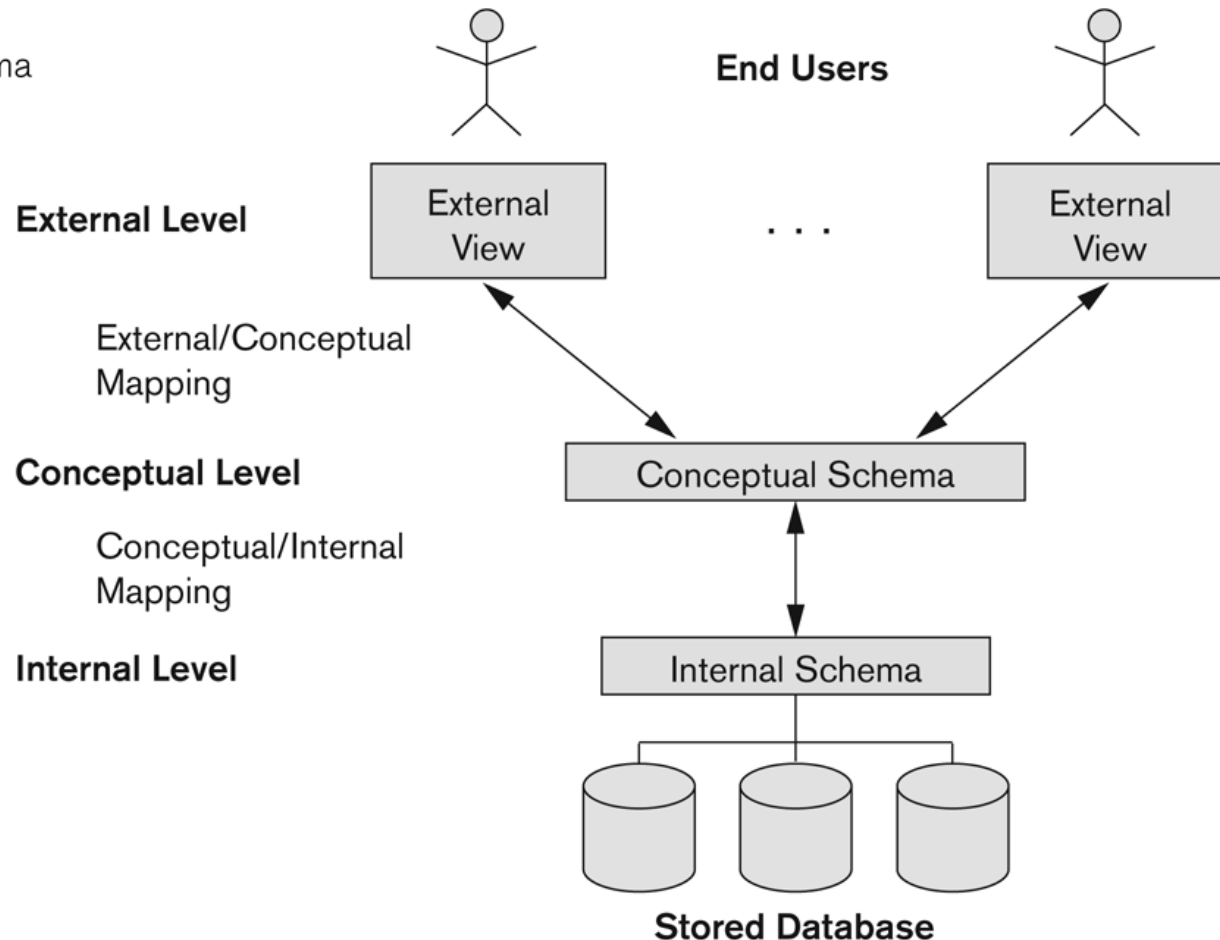
Figure 1.2

A database that stores student and course information.

The three-schema architecture

Figure 2.2

The three-schema architecture.



External Level

- Represents how end users see the data.
- Each user (or application) can have a different view of the same database.
- Shows only required data, hides the rest (security & simplicity).
- Example:
- Student sees: Name, GPA
- Admin sees: Name, GPA, Fees, Address

Conceptual Level

- Represents the logical structure of the entire database.
- Describes:
 - Entities
 - Attributes
 - Relationships
 - Constraints

Designed by the **database designer**.

STUDENT(StudentID, Name, GPA)

COURSE(CourseID, Title)

ENROLLS(StudentID, CourseID)

Internal Level

- Describes how data is physically stored.
- Includes:
 - File organization
 - Indexes
 - Storage structures

Hidden from users.

DBMS Languages

- Data Definition Language (DDL)
 - Used to define and modify the database structure.
 - Specifies schemas, tables, attributes, and constraints.
 - Affects the **database schema**, not the actual data.
 - Examples:
- CREATE
- ALTER
- DROP

DBMS Languages

- Data Manipulation Language (DML)
 - Used to retrieve and manipulate data stored in the database.
 - Works on the database instance (data).
 - Examples of DML commands:
 - SELECT
 - INSERT
 - UPDATE
 - DELETE

Types of DML

■ High Level or Non-procedural Language:

- Specify WHAT data is required, not HOW to get it. For example, the SQL relational language
- The DBMS chooses the execution strategy.
- Also called **declarative** languages.

Can be:

- Used standalone

```
SELECT * FROM STUDENT WHERE GPA > 3.0;
```

- Embedded in a host programming language

```
C: EXEC SQL SELECT Name INTO :name FROM  
STUDENT WHERE ID = :id;
```

Types of DML

- **Low Level or Procedural Language:**

- Specify HOW to access and manipulate data step by step.
- Must be embedded in a programming language.
- Programmer controls the procedure.

- **Examples: PL/SQL,**

BEGIN

UPDATE STUDENT SET GPA = 3.5 WHERE ID = 10;

END;

DBMS Interfaces

- Stand-alone query language interfaces
 - Example: Entering SQL queries at the DBMS interactive SQL interface (e.g. SQL*Plus in ORACLE)
- User-friendly interfaces
 - Menu-based, forms-based, graphics-based, etc.



- [Interface Configuration](#)
- System Configuration
 - [Script Formatting](#)
 - [Script Execution](#)
 - [Database Administration](#)

• **Change Password**

Change Password

Indicates required field

Cancel

Apply

* Username

* Old Password

* New Password

* Retype New Password

Cancel

Apply

[Workspace](#) | [History](#) | [Logout](#) | **[Preferences](#)** | [Help](#)**Oracle SQL*Plus**

File Edit Search Options Help

SQL*Plus: Release 8.0.4.0.0 - Production on Wed Nov 19 14:56:33 1997

(c) Copyright 1997 Oracle Corporation. All rights reserved.

Connected to:

Oracle8 Enterprise Edition Release 8.0.4.0.0 - Production

With the Partitioning and Objects options

PL/SQL Release 8.0.4.0.0 - Production

SQL>

DBMS Programming Language Interfaces

- Programmer interfaces for embedding DML in a programming languages:
 - **Embedded Approach:** e.g. embedded SQL (for C, C++, etc.), SQLJ (for Java)
 - **Procedure Call Approach:** e.g. JDBC for Java, ODBC for other programming languages
 - **Database Programming Language Approach:** e.g. ORACLE has PL/SQL, a programming language based on SQL;

DBMS Programming Language Interfaces

- **Procedure Call Approach:** SQL is not written directly in the program.
- Instead, the program calls database APIs that send SQL commands to the DBMS.
- Examples:
 - JDBC (Java Database Connectivity)
 - ODBC (Open Database Connectivity)
- `ResultSet rs = stmt.executeQuery(`
- `"SELECT name FROM student WHERE id = 10");`
- The DBMS still decides HOW to execute the query.

Centralized and Client-Server DBMS Architectures

■ Centralized DBMS:

- Combines everything into single system including- DBMS software, hardware, application programs, and user interface processing software.
- User can still connect through a remote terminal – however, all processing is done at centralized site.
- Examples?

A Physical Centralized Architecture

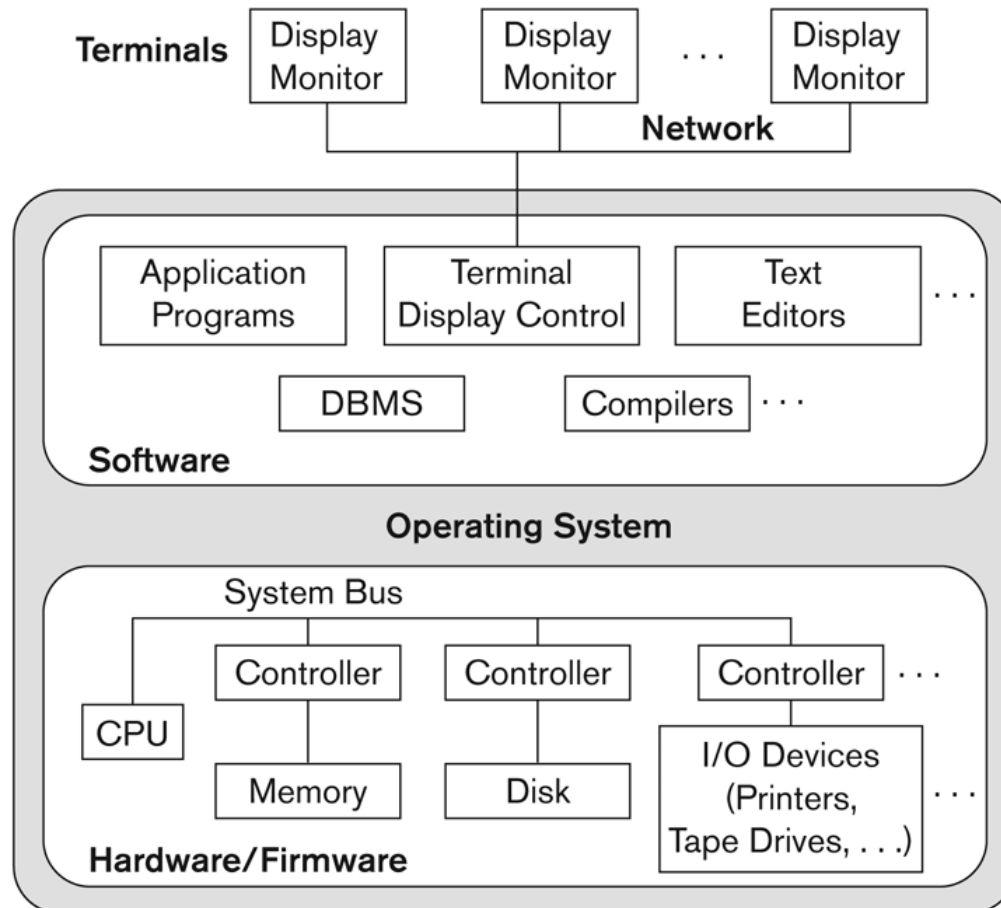


Figure 2.4
A physical centralized architecture.

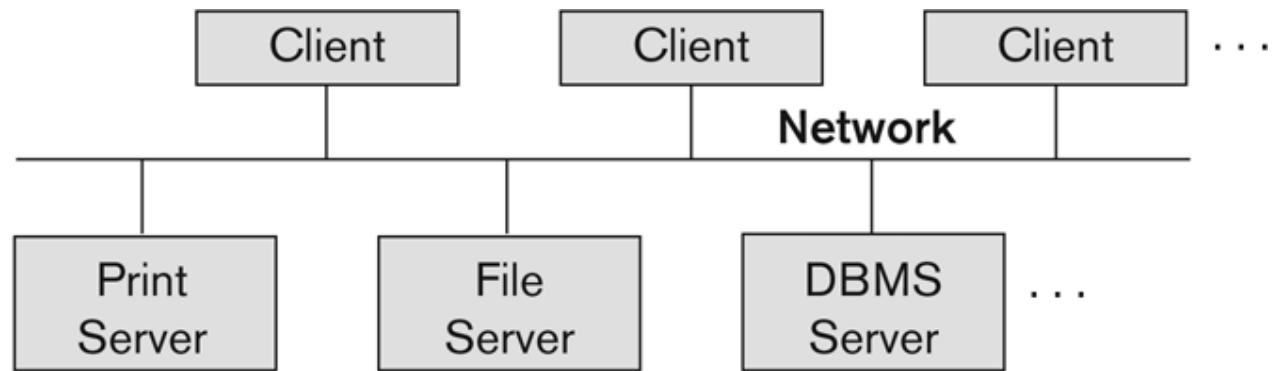
Basic 2-tier Client-Server Architectures

- Specialized Servers with Specialized functions
 - Print server
 - File server
 - DBMS server
 - Web server
 - Email server
- Clients can access the specialized servers as needed

Logical two-tier client server architecture

Figure 2.5

Logical two-tier
client/server
architecture.



Clients

- Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- Connected to the servers via some form of a network.
 - (LAN: local area network, wireless network, etc.)

Three Tier Client-Server Architecture

- Common for Web applications
- Intermediate Layer called Application Server or Web Server:
- Three-tier Architecture Can Enhance Security:
 - Database server only accessible via middle tier
 - Clients cannot directly access database server

Three-tier client-server architecture

Figure 2.7

Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.

